ST 538 Group 4 Project 2

Paul Anderson, Michael Barton, Kevin Kincaid

Dataset Background

Our project is focused on predicting whether a customer for a Vehicle Loan will default based on their financial state at time of borrowing. Default Vehicle loan will default prediction presents a distinct set of challenges relative to other retail credit products. Unlike mortgage or personal loan portfolios, two-wheeler financing in emerging markets such as India serves a large population of first-time borrowers with little or no bureau history, making traditional score-based underwriting insufficient on its own. This analysis uses the CRIF High Mark vehicle loan dataset to develop a feature set capable of identifying first-EMI default risk across both scored and thin-file borrower segments.

This analysis uses the CRIF High Mark vehicle loan dataset to develop a feature set capable of identifying first-EMI default risk across both scored and thin-file borrower segments. The dataset comprises 233,154 disbursed loans with a binary default indicator capturing payment failure on the first EMI due date. A notable characteristic of this population is that 56% of borrowers carry no scoreable bureau history, which necessitates treating thin-file and scored segments with care rather than imputing or ignoring the distinction. Among scored borrowers, the CRIF CNS score serves as the single strongest individual predictor and is encoded here as a 20-level ordered factor preserving the full risk ranking across bands.

Statistical Analysis

Feature extraction and description

Thirty-five percent of loans are high loan to value (LTV) of >80%, combined with a large portion of borrowers having thin credit history (%), it shows a meaningful concentration of risk. The vast majority (94%) of borrowers fall in the prime age band, 13,812 young borrowers (<22 years of age) are worth watching. Fifty-six percent of borrowers are “thin file” meaning they have <12 month of credit history. This makes sense, in India about half of first time car buyers are using lending for the first time. Since thin and normal are two different populations,

we debated on splitting the dataset and creating a model for thin credit and one for normal. Ultimately we decided it would be better to focus our efforts on normal only as the model will likely suffer trying to work on both.

After initial examination of this dataset, we were again reminded of the high dimensionality of each observation. We wanted to be able to focus on important features. Given the size of the dataset will already impact the speed of analytical methods, we want to remove as many unnecessary variables as possible. Additionally, there are several columns with missing values or continuous variables that would serve better on translating into categories.

As such, our feature engineering focused on translating domain knowledge into model-ready variables: loan structure risk (LTV, implied down payment), borrower demographics (age at disbursement, employment type), identity verification depth (KYC document count), credit utilisation and delinquency history across primary and secondary accounts, EMI burden relative to existing obligations, and recent bureau activity including new account uptake and hard inquiries.

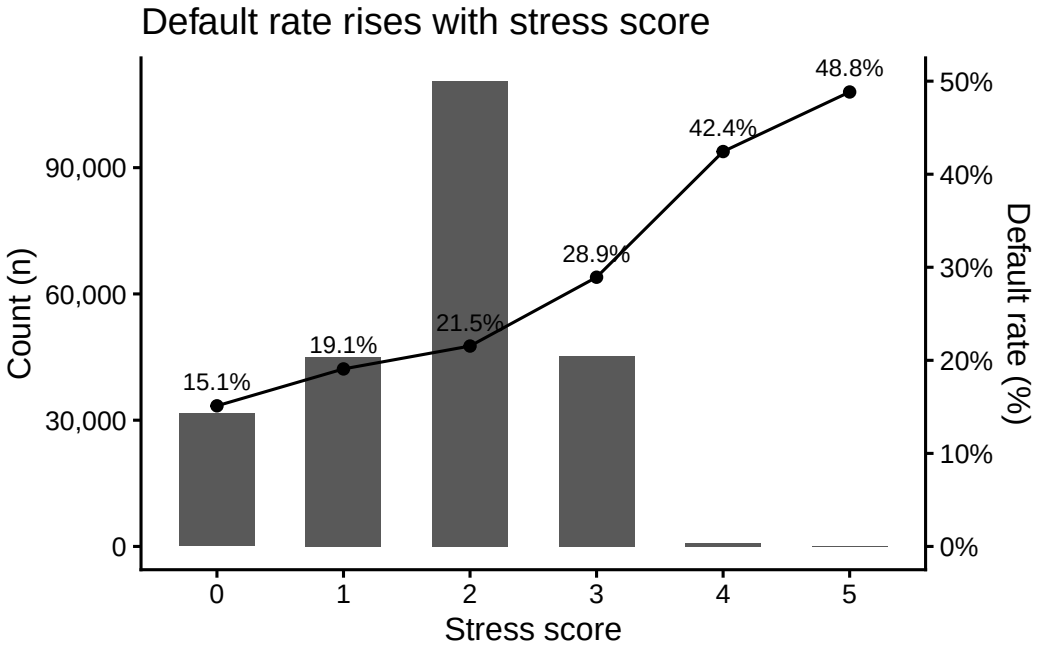
Table 1: Default Rate by Stress Score

Stress score	Total (n)	No default	Default	Default rate
0	31,573	26,802	4,771	15.1%
1	44,945	36,371	8,574	19.1%
2	110,556	86,748	23,808	21.5%
3	45,181	32,110	13,071	28.9%
4	813	468	345	42.4%
5	86	44	42	48.8%

To consolidate the risk signals across categories, we constructed a composite stress score by summing six binary flags, each capturing a distinct dimension of borrower risk. The first two flags address active repayment failures: whether the borrower has at least one overdue primary account, and whether they carry overdue exposure on a secondary account where they appear as co-applicant or guarantor, a contingent liability that standard underwriting often underweights. The third flag captures recent behavioral signal, marking any borrower who experienced an account delinquency in the six months prior to disbursement. The remaining three flags address structural vulnerability in the borrower's financial position: whether loan-to-value exceeds 80%, leaving minimal equity cushion in the asset, whether the borrower has fewer than 12 months of credit history, and whether their bureau description falls into one of seven unscoreable categories, meaning no reliable credit score can be assigned at all.

The additive structure is intentional. Rather than weighting any single factor, the score reflects simultaneous stress across multiple dimensions. A borrower with a high LTV alone is a manageable risk, a borrower who is also recently delinquent, has no credit history, and carries no bureau score represents a fundamentally different profile. The score ranges from 0 to 6, and

we additionally derived high_stress_flag as a binary indicator for scores of 3 or more. Initial cross-tabulation confirms that the stress score tracks default rate monotonically, rising from 15% at score 0 to 49% at score 5, nearly a 3x lift, providing early evidence that the engineered features carry meaningful signal.



The figure confirms that the stress score functions as an effective risk signal. Default rate rises monotonically from 15% at score 0 to 49% at score 5, a threefold increase across the range. The majority of borrowers fall at score 2, where the default rate of 21.5% closely tracks the overall population rate, reflecting the modal profile of a borrower who is thin-file and high-LTV but has no active delinquency. The steepest step in the gradient occurs between scores 2 and 3, where the default rate jumps nearly 8 percentage points, suggesting that the primary overdue and recent delinquency flags carry disproportionate weight once a borrower already carries structural risk. Scores 4 and 5 represent a small combined segment of under 900 borrowers but carry default rates of 42% and 49% respectively, and are retained in the feature set given their strong signal. Notably, even score 0 borrowers default at a rate of 15%, a reminder that first-EMI default in this market reflects payment intent as much as financial capacity. While the stress score is included as a standalone feature, the individual component flags are also retained, as tree-based models may capture non-linear interactions among them that the additive score cannot express.

Table 2: Feature Index Table

Category	Features
Bureau score	cns_score_factor, thin_file_flag, thin_file_reason, cns_score_clean

Category	Features
Composite	stress_score, high_stress_flag
Demographics	age_at_disbursal, age_segment, disbursal_month, disbursal_quarter, disbursal_year, fy_
EMI / DSCR	total_emi_burden, new_loan_emi_ratio
KYC	kyc_doc_count, strong_kyc_flag
Loan terms	ltv_high_flag, down_payment_amt, down_payment_pct
Primary credit	pri_active_util_ratio, pri_any_overdue_flag, pri_leverage_ratio, pri_avg_loan_size, pri_
Recent activity	recent_delinquency_flag, inquiry_to_new_acct_ratio, avg_acct_age_months, credit_hist
Secondary credit	sec_any_overdue_flag, sec_has_exposure_flag

Predictive Model Fitting

After feature extraction and variable selection, the dataset was split into training, validation, and testing subsets using a 70/15/15 ratio. Given the decision to model scored borrowers separately, the split was applied to the 103,369 non-thin-file records, resulting in approximately 72,000 observations in the training set and roughly 15,000 each in the validation and testing sets. Two classification methods were evaluated: Random Forest and XGBoost.

Prior to model fitting, columns were removed from the feature set for one of three reasons: missingness, redundancy, or lack of predictive relevance. Three columns contained missing values incompatible with random forest. EMPLOYMENT_TYPE carried 7,661 NAs from blank strings in the raw data, thin_file_reason carried 103,369 NAs by design as it is only populated for thin-file borrowers, and new_loan_emi_ratio carried 159,517 NAs arising when PRIMARY_INSTAL_AMT is zero. Rather than impute these, they were excluded given that the information each carries is represented elsewhere in the feature set.

Administrative identifiers with no predictive signal were also removed, including UNIQUEID, BRANCH_ID, EMPLOYEE_CODE_ID, and MANUFACTURER_ID. DISBURSAL_DATE and STATE_ID were excluded as their relevant information had already been extracted into engineered features during feature construction.

Finally, the four weakest KYC flags were dropped in favor of the summary measures kyc_doc_count and strong_kyc_flag, and the secondary account financial detail columns were removed as redundant given the derived exposure flags already in the feature set. LOAN_DEFAULT was excluded as the response variable.

I THINK THE ABOVE MIGHT GO ON FOR TWO LONG, especially as we do a lot of other stuff in this section

First, we tried fitting a default random forest classifier to the training data and evaluating its performance on both the training and validation datasets. Due to the strong class imbalance in the loan default variable, the initial models showed poor sensitivity for detecting loan defaults,

despite relatively high overall accuracy. The majority of the mis-classifications occurred on loan defaults.

Running diagnostics on the default random forest model, we observed that the variable importance results appeared distorted by random forest's default Gini importance metric (Mean Decrease Gini), which favors high-cardinality variables (CURRENT_PINCODE_ID, DATE_OF_BIRTH, DISBURSED_AMOUNT) because they offer more split opportunities, not necessarily because they better predict defaults. Meanwhile, credit-risk variables that should be more informative (PERFORM_CNS_SCORE, delinquency measures, leverage ratio, shown in blue) rank in the middle, likely due to class imbalance rather than a lack of predictive value. We used this information to adjust the switch the importance metric to Mean Decrease Accuracy, along with a balanced sub-sample of each group, and ran again.

Table 3: Random Forest Performance Metrics

Model	Dataset	Accuracy	Recall	Precision	Specificity	F1
rf.sim2	Training	66.8%	56.1%	31.8%	69.5%	40.6%
rf.sim2	Validation	66.2%	55.5%	30.8%	68.8%	39.6%
rf.sim2	Test	66.5%	57.7%	30.9%	68.7%	40.3%

The resulting model showed a substantially better balance between correctly identifying defaulting and non-defaulting loans. On the validation dataset, the model achieved an accuracy of 66.2%, a recall of 55.4%, and a specificity of 68.8%, indicating that it was able to correctly identify more than half of all loan defaults while maintaining reasonable performance on non-defaulting loans. Precision remained relatively low at 30.8%, reflecting the continued challenge of accurately identifying defaults in an imbalanced dataset. Similar results were observed on the test dataset, where the model achieved an accuracy of 66.5%, a recall of 57.7%, and a specificity of 68.7%. The close agreement between validation and test performance suggests that the model generalized well to unseen data and was not substantially overfit to the training set. Overall, the tuned random forest represented a meaningful improvement over the baseline model by substantially increasing default detection while maintaining stable performance across datasets.

Table 4: Random Forest Model Comparison

Model	Dataset	TN	FP	FN	TP	Accuracy	Recall	Precision	Specificity	F1
rf.sim1	Training	57124	395	14256	355	0.797	0.024	0.473	0.993	0.046
rf.sim1	Validation	12415	94	3056	78	0.799	0.025	0.453	0.992	0.047
rf.sim2	Training	39958	17561	6416	8195	0.668	0.561	0.318	0.695	0.406

Model	Dataset	TN	FP	FN	TP	Accuracy	Recall	Precision	Specificity	F1
rf.sim2	Validation	8613	3896	1397	1737	0.662	0.554	0.308	0.689	0.396
rftune	Training	43542	13977	7518	7093	0.702	0.485	0.337	0.757	0.398
rftune	Validation	9345	3164	1646	1488	0.693	0.475	0.320	0.747	0.382

We also explored additional tuning strategies including the use of tuneRF() to optimize the mtry parameter and balanced sampling to improve classification performance. The tuned model substantially improved recall for default predictions, though it also increased the number of FALSE positive classifications. While the tuneRf random forest model showed meaningful improvement over the baseline models (Table 2), prediction performance for loan defaults remained moderate, indicating that additional tuning or alternative classification methods may be necessary for highly imbalanced loan data.

```
conf_xgb <- matrix(
  c(11792, 717,
    2971, 163),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Actual = c("0", "1"),
    Predicted = c("0", "1")
  )
)

knitr::kable(
  conf_xgb,
  caption = "XGBoost Validation Confusion Matrix"
)
```

Table 5: XGBoost Validation Confusion Matrix

	0	1
0	11792	717
1	2971	163

```r
xgb_threshold_results <- tribble(
  ~Threshold, ~Accuracy, ~Recall, ~Precision, ~Specificity, ~F1,
  0.10, 0.2075050, 0.9875558, 0.2002847, 0.0120713, 0.3330285,
  0.15, 0.2922074, 0.8567326, 0.2017583, 0.1507714, 0.3266026,
  0.20, 0.4821965, 0.5402042, 0.2027059, 0.4676633, 0.2947937,
  0.25, 0.6723135, 0.2115507, 0.1998192, 0.7877528, 0.2055177,
  0.30, 0.7642396, 0.0520102, 0.1852273, 0.9426813, 0.0812157
)

xgb_threshold_results %>%
  mutate(
    across(
      c(Accuracy, Recall, Precision, Specificity, F1),
      ~ scales::percent(.x, accuracy = 0.1)
    )
  ) %>%
  kable(
    caption = "XGBoost Performance by Classification Threshold"
  )
```

Table 6: XGBoost Performance by Classification Threshold

Threshold	Accuracy	Recall	Precision	Specificity	F1
0.10	20.8%	98.8%	20.0%	1.2%	33.3%
0.15	29.2%	85.7%	20.2%	15.1%	32.7%
0.20	48.2%	54.0%	20.3%	46.8%	29.5%
0.25	67.2%	21.2%	20.0%	78.8%	20.6%
0.30	76.4%	5.2%	18.5%	94.3%	8.1%

Finally, we trained an XGBoost model to explore if alternative classifiers could improve on the results we were observing in our random forest model. We utilized the scale_pos_weight parameter when training the model to try compensating for the imbalance in the response variable. After training the model still had low accuracies, with a tested AUC score of 0.50. To further evaluate XGBoost classifier performance, we examined the effect of varying the probability threshold used for default classification. We found that lower thresholds substantially increased recall, identifying nearly all default cases, but at the cost of a large number of false positives. Conversely, higher thresholds improved specificity while dramatically reducing the model's ability to detect defaults. Notably, precision remained relatively constant across all thresholds, suggesting that loans assigned higher default probabilities were not substantially more likely to default than those assigned lower probabilities. This finding is consistent with the model's AUC of 0.50, indicating that the XGBoost classifier provided only marginal

improvement over random classification and was unable to effectively separate defaulting and non-defaulting loans using the available predictors.

Comparison of Predicted Results

```
conf_rf1_test <- matrix(
  c(8567, 3978,
    1269, 1782),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Actual = c("0", "1"),
    Predicted = c("0", "1")
  )
)

knitr::kable(
  conf_rf1_test,
  caption = "Random Forest Model 1 Test Set Confusion Matrix"
)
```

Table 7: Random Forest Model 1 Test Set Confusion Matrix

	0	1
0	8567	3978
1	1269	1782

```
# Static table of test Data on random forest 2 (Paul's improved model)
conf_rf2_test <- matrix(
  c(8618, 3927,
    1292, 1759),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Actual = c("0", "1"),
    Predicted = c("0", "1")
  )
)
```

```
knitr::kable(
  conf_rf2_test,
  caption = "Random Forest Model 2 Test Set Confusion Matrix"
)
```

Table 8: Random Forest Model 2 Test Set Confusion Matrix

	0	1
0	8618	3927
1	1292	1759

```
# static table of test data on XGB
conf_xgb_test <- matrix(
  c(11731, 814,
    2902, 149),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Actual = c("0", "1"),
    Predicted = c("0", "1")
  )
)

knitr::kable(
  conf_xgb_test,
  caption = "XGBoost Test Set Confusion Matrix"
)
```

Table 9: XGBoost Test Set Confusion Matrix

	0	1
0	11731	814
1	2902	149

Obstacles

There are several complications with this dataset. One is in the variety of data. Files marked as “thin file” with little credit history had different demographic patterns and higher default

rates, enough to the point we considered splitting the dataset into two populations. We tried to compensate with the steps we took in feature engineering. There was also the size of the dataset. The sheer number of observations means that running our analysis took significantly longer as certain methods take five minutes to compile and check results. The size coupled with the dimensionality meant some functions relating to random forest couldn't be used at all on our devices. We compensated for this by consolidating features and using methods with LASSO to remove excess data before passing to functions.

Another issue we had was in the random forest prioritization of variables. Basedo on our assessement of variable importance, there were certain markers that should've been utilized as the most important. However, we determined that random forest was prioritizing variables with high cardinality. We exerpimented with switching the variable importance evaluation method which proved useful.

The most major issue we encountered was due to the unbalanced data, as there a much higher number of defaults compared to non-defaults. The exact ratio is . Different models we trained on the data handled the different categorizes of data inconsistently. Random forest would make accurate non-default forecasting, but would perform poorly on loans that defaulted. XGBoost at the standard threshold performed well on loans that defaulted, but not on loans that didn't. We had decided to use SMOTE methods, as those are commonly suggested as a mitigator for unbalanced data. However, when we implemented this in our dataset it only marginally decreased our prediction errors. One possible issue might have been the size of our dataset and it's imbalance might have limited this methods effectiveness.

Conclusion

In conclusion, while it is possible to fit many different models to our data, we can see how hard in practice it is to make an effective model, especially in situations with large datasets and imbalanced categories. As is inherent, there is a much smaller number of defaulted borrowers, but that disproportionate amounts means it is harder to utilize methods to accurate predict whether any specific borrowers will default. Different methods had different accuracy between defaults and non-defaults. We did get to experiment with a number of techniques in this project but ultimately none of them really improved the results to the expected level.

Next Steps

There are several further steps we could take with our exploration. One potential idea was to fully treat Thin File and normal file as separate populations, and try training two separate models for each and seeing if we are able to have better predictive accuracy

Appendix

```
# Write a report based on the statistical analysis in this project. As described in the proje
#
# Submit a four-page (reasonable margins and font size) document that summarizes the elements
#
# Project/data background and the questions that you addressed.
# Statistical analysis reported in the same framework as in interim reports -
# -  feature selection,
# -  predictive model fitting,
# -  comparison of prediction results of the fitted models, specifically on the held-out test
# -  Obstacles you encountered and useful/interesting solutions to overcoming those obstacles
# -  Conclusion with additional questions that you generated and/or future work that you coul
#
# Submit the completed report as a link in the relevant discussion.
# The report should be at most four pages, not counting figures and R codes. Please include t
# load dependencies
library(tidyverse)
library(lubridate)
library(stringr)
library(knitr)
library(randomForest)

# Week 3 (report 2)
#install.packages("xgboost")
library(xgboost)
library(caret)
library(pROC)
# install.packages("kable")
# library(kable)
# library(kableExtra)
# Project URL https://www.kaggle.com/datasets/avikpaul4u/vehicle-loan-default-prediction?sele

# Load our files (saved manually to data directory)
# Training data
train_dat = read.csv('./data/train.csv')

# Testing data (DO NOT LOAD UNTIL WEEK 8 OR 9!!)
# Also, the test.csv does not have Loan_Default, so it may not work for this
#test_dat = read.csv('./data/test.csv')

head(train_dat)
```

```r
str(train_dat)
# Check for any columns with missing data
sapply(train_dat, anyNA)

# There are no missingness in the data! Pretty sweet!
# helper function - parse tenure string to numeric months
parse_tenure_months <- function(x) {
  years  <- as.numeric(str_extract(x, "\\d+(?=yrs)"))
  months <- as.numeric(str_extract(x, "\\d+(?=mon)"))
  years  <- ifelse(is.na(years),  0, years)
  months <- ifelse(is.na(months), 0, months)
  years * 12 + months
}

# loan terms
train_dat <- train_dat %>%
  mutate(

    # LTV risk flag - regulatory / underwriting threshold
    ltv_high_flag = as.integer(LTV > 80),

    # Implied down payment
    down_payment_amt = ASSET_COST - DISBURSED_AMOUNT,
    down_payment_pct = round((down_payment_amt / ASSET_COST) * 100, 2)

  )

# demographics and disbursement date
train_dat <- train_dat %>%
  mutate(

    # Parse dates - format is "DD-MM-YYYY" per the raw data
    dob_parsed     = dmy(DATE_OF_BIRTH),
    disbursal_date = dmy(DISBURSAL_DATE),

    # Age at disbursement in whole years
    age_at_disbursal = as.integer(
      interval(dob_parsed, disbursal_date) / years(1)
    ),

    # Age risk segments
    age_segment = case_when(
```

```
      age_at_disbursal <  22                              ~ "young_risky",
      age_at_disbursal >= 22 & age_at_disbursal <= 60     ~ "prime",
      age_at_disbursal >  60                              ~ "senior_risky",
      TRUE                                                ~ NA_character_
    ),

    # Disbursement seasonality
    disbursal_month   = month(disbursal_date),
    disbursal_quarter = quarter(disbursal_date),
    disbursal_year    = year(disbursal_date),

    # Indian fiscal year-end flag (Jan-Mar = Q4 of Indian FY)
    fy_end_flag = as.integer(disbursal_month %in% c(1, 2, 3))

  ) %>%
  select(-dob_parsed)   # intermediate parse column no longer needed

# identity
train_dat <- train_dat %>%
  mutate(

    # Total KYC documents submitted (0-6)
    kyc_doc_count = MOBILENO_AVL_FLAG + AADHAR_FLAG + PAN_FLAG +
                    VOTERID_FLAG + DRIVING_FLAG + PASSPORT_FLAG,

    # Strong KYC: has both Aadhaar (biometric) AND PAN (income tax records)
    strong_kyc_flag = as.integer(AADHAR_FLAG == 1 & PAN_FLAG == 1)

  )

# bureau score

# sort(unique(train_dat$PERFORM_CNS_SCORE_DESCRIPTION)) # check the label strings

# CRIF CNS bands A-M run lowest to highest risk.
# "Not Scored" has 6 distinct reason codes - all treated as thin-file.
# Ordered worst -> best creditworthiness so ordinal models can use the ranking.

cns_levels <- c(
  # --- Thin-file / unscoreable (placed at bottom = worst/unknown) ---
  "No Bureau History Available",
  "Not Scored: More than 50 active Accounts found",
```

```
  "Not Scored: No Activity seen on the customer (Inactive)",
  "Not Scored: No Updates available in last 36 months",
  "Not Scored: Not Enough Info available on the customer",
  "Not Scored: Only a Guarantor",
  "Not Scored: Sufficient History Not Available",
  # --- Scored bands: highest risk first ---
  "M-Very High Risk",
  "L-Very High Risk",
  "K-High Risk",
  "J-High Risk",
  "I-Medium Risk",
  "H-Medium Risk",
  "G-Low Risk",
  "F-Low Risk",
  "E-Low Risk",
  "D-Very Low Risk",
  "C-Very Low Risk",
  "B-Very Low Risk",
  "A-Very Low Risk"
)

# All "Not Scored" reason strings for thin-file flagging
not_scored_reasons <- c(
  "No Bureau History Available",
  "Not Scored: More than 50 active Accounts found",
  "Not Scored: No Activity seen on the customer (Inactive)",
  "Not Scored: No Updates available in last 36 months",
  "Not Scored: Not Enough Info available on the customer",
  "Not Scored: Only a Guarantor",
  "Not Scored: Sufficient History Not Available"
)

train_dat <- train_dat %>%
  mutate(

    # Ordered factor - preserves risk ranking in tree-based and ordinal models
    cns_score_factor = factor(
      PERFORM_CNS_SCORE_DESCRIPTION,
      levels  = cns_levels,
      ordered = TRUE
    ),
```

```
    # Thin-file flag: covers all 7 unscoreable categories
    thin_file_flag = as.integer(
      PERFORM_CNS_SCORE_DESCRIPTION %in% not_scored_reasons
    ),

    # Thin-file reason - retains the specific not-scored reason for analysis
    # (e.g. "Only a Guarantor" may have a different risk profile than "Inactive")
    thin_file_reason = ifelse(
      thin_file_flag == 1,
      PERFORM_CNS_SCORE_DESCRIPTION,
      NA_character_
    ),

    # Clean score: NA out thin-file records so 0 is not treated as a real score
    cns_score_clean = ifelse(thin_file_flag == 1, NA_real_, PERFORM_CNS_SCORE)
    )

# primary accounts
train_dat <- train_dat %>%
  mutate(

    # Active loan utilisation ratio
    pri_active_util_ratio = ifelse(
      PRI_NO_OF_ACCTS > 0,
      round(PRI_ACTIVE_ACCTS / PRI_NO_OF_ACCTS, 4),
      0
    ),

    # Any overdue primary account - binary red flag
    pri_any_overdue_flag = as.integer(PRI_OVERDUE_ACCTS > 0),

    # Incremental leverage: existing balance relative to new loan size
    pri_leverage_ratio = ifelse(
      DISBURSED_AMOUNT > 0,
      round(PRI_CURRENT_BALANCE / DISBURSED_AMOUNT, 4),
      NA_real_
    ),

    # Average primary loan size historically
    pri_avg_loan_size = ifelse(
      PRI_NO_OF_ACCTS > 0,
      round(PRI_DISBURSED_AMOUNT / PRI_NO_OF_ACCTS, 2),
```

```
      0
    ),

    # Sanctioned vs. disbursed gap (large gap may indicate cancellations)
    pri_sanction_disbursed_gap = PRI_SANCTIONED_AMOUNT - PRI_DISBURSED_AMOUNT

  )

# joint accounts
train_dat <- train_dat %>%
  mutate(

    # Binary: any secondary overdue
    sec_any_overdue_flag  = as.integer(SEC_OVERDUE_ACCTS > 0),

    # Has any secondary exposure at all
    sec_has_exposure_flag = as.integer(SEC_NO_OF_ACCTS > 0)

  )

# EMI / debt service
train_dat <- train_dat %>%
  mutate(

    # Total monthly EMI obligation across primary and secondary loans
    total_emi_burden = PRIMARY_INSTAL_AMT + SEC_INSTAL_AMT,

    # New loan EMI relative to existing primary EMI commitment
    # Ratio > 1: new loan eclipses all current EMI obligations combined
    new_loan_emi_ratio = ifelse(
      PRIMARY_INSTAL_AMT > 0,
      round(DISBURSED_AMOUNT / PRIMARY_INSTAL_AMT, 4),
      NA_real_
    )

  )

# recent bureau activity
train_dat <- train_dat %>%
  mutate(

    # Binary: any delinquency in the last 6 months
```

```
    recent_delinquency_flag = as.integer(DELINQUENT_ACCTS_IN_LAST_SIX_MONTHS > 0),

    # Inquiry-to-new-account ratio
    # When NEW_ACCTS = 0: return raw inquiry count - all enquiries were rejected
    inquiry_to_new_acct_ratio = ifelse(
      NEW_ACCTS_IN_LAST_SIX_MONTHS > 0,
      round(NO_OF_INQUIRIES / NEW_ACCTS_IN_LAST_SIX_MONTHS, 4),
      NO_OF_INQUIRIES
    ),

    # Parse "Xyrs Ymon" tenure strings to numeric months
    avg_acct_age_months    = parse_tenure_months(AVERAGE_ACCT_AGE),
    credit_history_months = parse_tenure_months(CREDIT_HISTORY_LENGTH),

    # Thin credit history flag (<12 months = thin-file risk)
    short_history_flag = as.integer(credit_history_months < 12)

  )

# composite financial stress score
train_dat <- train_dat %>%
  mutate(

    # Additive count of binary risk flags
    # Useful for EDA, stakeholder comms, and model sanity checks
    stress_score = pri_any_overdue_flag    +
                   sec_any_overdue_flag    +
                   recent_delinquency_flag +
                   ltv_high_flag           +
                   short_history_flag      +
                   thin_file_flag,

    # High-stress flag: 3 or more simultaneous risk indicators
    high_stress_flag = as.integer(stress_score >= 3)

  )

# employment type na results fix
train_dat <- train_dat %>%
    mutate(EMPLOYMENT_TYPE = na_if(EMPLOYMENT_TYPE, ""))
  # #head(train_dat)
  # #add graphs to demonstrate everything in the paragraph below
```

```
  #
  # #high loan to value
  # #thin file defaults
  # #age bands default vs no default
  # library(ggplot2)
  # train_dat |>
  #   filter(LOAN_DEFAULT == 1) |>
  #   ggplot(aes(factor(age_at_disbursal), LTV)) +
  #   geom_boxplot(color = 'red') +
  #   labs(x = "Age at Dispersal", y = "Distribution of Loan-to-Value") +
  #   ggtitle("LTV Distribution by Age for Defaulted Loans") +
  #   theme(plot.title = element_text(hjust = 0.5))
  #
  # train_dat |>
  #   filter(LOAN_DEFAULT == 0) |>
  #   ggplot(aes(factor(age_at_disbursal), LTV)) +
  #   geom_boxplot(color = 'blue')+
  #   labs(x = "Age at Dispersal", y = "Distribution of Loan-to-Value") +
  #   ggtitle("LTV Distribution by Age for Active Loans") +
  #   theme(plot.title = element_text(hjust = 0.5))
  #
  # train_dat |>
  #   filter(LOAN_DEFAULT == 1) |>
  #   ggplot(aes(credit_history_months)) +
  #   geom_histogram(bins = 30) +
  #   labs(x = "Months of Credit History", y = "Number of Defaulted Borrowers") +
  #   ggtitle("Defaulted Borrowers by Months of Credit History") +
  #   theme(plot.title = element_text(hjust = 0.5))

# validation section - set include to TRUE for EDA exploration
cat("Rows:", nrow(train_dat), "\n")
cat("Total columns after engineering:", ncol(train_dat), "\n")
cat("Net new columns:", ncol(train_dat) - 41, "\n\n")

cat("--- LTV high flag (>80%) ---\n")
print(table(train_dat$ltv_high_flag, useNA = "ifany"))

cat("\n--- Age segment ---\n")
print(table(train_dat$age_segment, useNA = "ifany"))

cat("\n--- Employment type ---\n")
print(table(train_dat$EMPLOYMENT_TYPE, useNA = "ifany"))
```

```r
cat("\n--- CNS score factor levels (confirm all levels matched) ---\n")
print(levels(train_dat$cns_score_factor))
cat("Unmatched descriptions (NAs in factor):",
    sum(is.na(train_dat$cns_score_factor)), "\n")

cat("\n--- Thin-file borrowers ---\n")
print(table(train_dat$thin_file_flag, useNA = "ifany"))

cat("\n--- Primary any overdue ---\n")
print(table(train_dat$pri_any_overdue_flag, useNA = "ifany"))

cat("\n--- Recent delinquency flag ---\n")
print(table(train_dat$recent_delinquency_flag, useNA = "ifany"))

cat("\n--- Stress score distribution ---\n")
print(table(train_dat$stress_score, useNA = "ifany"))

cat("\n--- avg_acct_age_months (first 10 rows) ---\n")
print(data.frame(
  raw    = head(train_dat$AVERAGE_ACCT_AGE, 10),
  parsed = head(train_dat$avg_acct_age_months, 10)
))

cat("\n--- credit_history_months (first 10 rows) ---\n")
print(data.frame(
  raw    = head(train_dat$CREDIT_HISTORY_LENGTH, 10),
  parsed = head(train_dat$credit_history_months, 10)
))
# default rate by stress score table
stress_table <- train_dat %>%
  group_by(stress_score) %>%
  summarise(
    n           = n(),
    n_default   = sum(LOAN_DEFAULT, na.rm = TRUE),
    .groups     = "drop"
  ) %>%
  mutate(
    n_no_default  = n - n_default,
    default_rate  = paste0(round(n_default / n * 100, 1), "%")
  ) %>%
  select(
    `Stress score`  = stress_score,
```

```
    `Total (n)`     = n,
    `No default`    = n_no_default,
    `Default`       = n_default,
    `Default rate`  = default_rate
  )

knitr::kable(
  stress_table,
  format      = "simple",
  caption     = "Default Rate by Stress Score",
  align       = c("c", "r", "r", "r", "r"),
  format.args = list(big.mark = ",")
)
# default rate by stress score graphically
stress_summary <- train_dat %>%
  group_by(stress_score) %>%
  summarise(
    n            = n(),
    n_default    = sum(LOAN_DEFAULT, na.rm = TRUE),
    .groups      = "drop"
  ) %>%
  mutate(default_rate = n_default / n * 100)

scale_factor <- max(stress_summary$n) / 50

ggplot(stress_summary, aes(x = factor(stress_score))) +

  geom_col(aes(y = n), width = 0.6) +

  geom_line(aes(y = default_rate * scale_factor, group = 1)) +
  geom_point(aes(y = default_rate * scale_factor)) +

  geom_text(
    aes(y = default_rate * scale_factor, label = paste0(round(default_rate, 1), "%")),
    vjust = -0.9, size = 3.2
  ) +

  scale_y_continuous(
    name     = "Count (n)",
    labels   = scales::comma,
    sec.axis = sec_axis(~ . / scale_factor, name = "Default rate (%)",
                        labels = scales::label_number(suffix = "%"))
```

```
  ) +

  theme_classic(base_size = 12) +
  theme(panel.grid.major.x = element_blank(),
        panel.grid.minor   = element_blank()) +

  labs(
    title = "Default rate rises with stress score",
    x     = "Stress score"
  )
# engineered feature index table - this can be pulled from if needed for context
feature_index <- tribble(
  ~Category,          ~Feature,
  "Loan terms",       "ltv_high_flag",
  "Loan terms",       "down_payment_amt",
  "Loan terms",       "down_payment_pct",
  "Demographics",     "age_at_disbursal",
  "Demographics",     "age_segment",
  "Demographics",     "disbursal_month",
  "Demographics",     "disbursal_quarter",
  "Demographics",     "disbursal_year",
  "Demographics",     "fy_end_flag",
  "KYC",              "kyc_doc_count",
  "KYC",              "strong_kyc_flag",
  "Bureau score",     "cns_score_factor",
  "Bureau score",     "thin_file_flag",
  "Bureau score",     "thin_file_reason",
  "Bureau score",     "cns_score_clean",
  "Primary credit",   "pri_active_util_ratio",
  "Primary credit",   "pri_any_overdue_flag",
  "Primary credit",   "pri_leverage_ratio",
  "Primary credit",   "pri_avg_loan_size",
  "Primary credit",   "pri_sanction_disbursed_gap",
  "Secondary credit", "sec_any_overdue_flag",
  "Secondary credit", "sec_has_exposure_flag",
  "EMI / DSCR",       "total_emi_burden",
  "EMI / DSCR",       "new_loan_emi_ratio",
  "Recent activity",  "recent_delinquency_flag",
  "Recent activity",  "inquiry_to_new_acct_ratio",
  "Recent activity",  "avg_acct_age_months",
  "Recent activity",  "credit_history_months",
  "Recent activity",  "short_history_flag",
```

```
  "Composite",        "stress_score",
  "Composite",        "high_stress_flag"
)

# collapse each category's features into a single comma-separated string
feature_index_wide <- feature_index %>%
  group_by(Category) %>%
  summarise(Features = paste(Feature, collapse = ", "), .groups = "drop")

knitr::kable(feature_index_wide, format = "simple", col.names = c("Category", "Features"),
             caption = "Feature Index Table")
# Splitting between thin file and normal
train_dat_normal <- train_dat %>%
  filter(thin_file_flag == 0)

train_dat_thin <- train_dat %>%
  filter(thin_file_flag == 1)

# Check counts of each
cbind(count(train_dat_normal),count(train_dat_thin))
set.seed(2026)

# Set our splits
train_percent <- 0.7
not_train_percent <- 0.3

# not_train split up
validate_percent <- 0.5 # Ends up being 0.15
test_percent <- 0.5 # Ends up being 0.15


# splitting into train/not_train -   this had an error with nrow(train_dat) producing too lo
sample <- sample(c(TRUE, FALSE), nrow(train_dat_normal), replace=TRUE, prob=c(train_percent,
training  <- train_dat_normal[sample, ]



not_training   <- train_dat_normal[!sample, ]

# splitting into validation/test
train_sample <- sample(c(TRUE, FALSE), nrow(not_training), replace=TRUE, prob=c(validate_per
validation  <- not_training[train_sample, ]
```

```r
testing   <- not_training[!train_sample, ]


rbind(count(training), count(validation), count(testing))

# # Did they default? Quick count check from our random sample
# train_dat |>
#   summarise(
#     defaulted_yes_overall = sum(LOAN_DEFAULT == 1),
#     defaulted_no_overall = sum(LOAN_DEFAULT == 0)
#
#   )
#
# train |>
#   summarise(
#     defaulted_yes_train = sum(LOAN_DEFAULT == 1),
#     defaulted_no_train = sum(LOAN_DEFAULT == 0)
#   )
#
# test |>
#   summarise(
#     defaulted_yes_test = sum(LOAN_DEFAULT == 1),
#     defaulted_no_test = sum(LOAN_DEFAULT == 0)
#
#   )

# Check number of NA's per column
sapply(train_dat, function(x) sum(is.na(x)))


 #which(names(training) %in% c("new_loan_emi_ratio","STATE_ID"))
  # the following have NA values which do not work with forest
  #10 EMPLOYMENT_TYPE
  #56 thin_file_reason

  #66 new_loan_emi_ratio

  #41 is predictor LOAN_DEFAULT
  #sapply(training[,c(-10, -56, -66, -42)], anyNA)

  #training |> filter(is.na(thin_file_reason))
  #training |> filter(is.na(high_stress_flag))
```

```r
# Columns that have NA's or are non-pertinent (UNIQUE_ID, MANUFACTURER_ID, etc)
drop_cols <- c(-1, -5, -7, -10, -11, -12, -13,
               -14, -17, -18, -19, -26, -27,
               -29, -31, -32, -33, -35,
               -56, -66, -41)

# Column numbers being dropped
drop_idx <- abs(drop_cols)

# Column names being dropped
names(train_dat)[drop_idx]


# head(training[drop_cols])

# Save the filtered data into new variables to make it easier
# but leave the original variables with all columns
x_training <- training[, drop_cols]
x_validation <- validation[, drop_cols]
x_testing <- testing[, drop_cols]

# NOTE: x_training, etc are not consistently used, instead I was using training[, drop_cols]
 rf.sim1 = randomForest(training[drop_cols],
                           factor(training$LOAN_DEFAULT))

    pred.rf.sim1 = rf.sim1$predicted
    table(training[,41], pred.rf.sim1)

   # varImpPlot(rf.sim1)
   #

   # #training |> filter(high_stress_flag == 1)
   #
   # #    pred.rf.sim1
   # #       0     1
   # # 0 57124   395
   # # 1 14256   355

# variable importance plot generated from rf.sim1
# gave this up in favor of the next chunk
#importance(rf.sim1) |>
#  as.data.frame() |>
```

```
#  rownames_to_column("Feature") |>
#  arrange(MeanDecreaseGini) |>
#  mutate(Feature = factor(Feature, levels = Feature)) |>
#  ggplot(aes(x = MeanDecreaseGini, y = Feature)) +
#  geom_col() +
#  theme_classic() +
#  labs(title = "RF1 Variable Importance", x = "Mean Decrease Gini", y = NULL)
# Theoretically meaningful variables - highlighted regardless of Gini rank
highlight_vars <- c(
  "PERFORM_CNS_SCORE",
  "PERFORM_CNS_SCORE_DESCRIPTION",
  "stress_score",
  "recent_delinquency_flag",
  "pri_any_overdue_flag",
  "DELINQUENT_ACCTS_IN_LAST_SIX_MONTHS",
  "credit_history_months",
  "avg_acct_age_months",
  "ltv_high_flag",
  "pri_leverage_ratio"
)

# Pull importance scores from model object
imp_df <- importance(rf.sim1) |>
  as.data.frame() |>
  rownames_to_column("Feature") |>
  arrange(MeanDecreaseGini) |>
  mutate(
    Feature    = factor(Feature, levels = Feature),
    Highlighted = ifelse(Feature %in% highlight_vars, "Key predictors", "Other")
  )

ggplot(imp_df, aes(x = MeanDecreaseGini, y = Feature, fill = Highlighted)) +
  geom_col() +
  scale_fill_manual(
    values = c("Key predictors" = "steelblue", "Other" = "grey75"),
    name   = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    axis.text.y      = element_text(size = 8),
    panel.grid.major.y = element_blank(),
    panel.grid.minor   = element_blank(),
```

```
    legend.position  = "top",
    plot.title       = element_text(size = 12)
  ) +
  labs(
    title = "RF1 Variable Importance",
    x     = "Mean Decrease Gini",
    y     = NULL
  )
rf.sim2 = randomForest(
  training[drop_cols],
  factor(training$LOAN_DEFAULT),
  sampsize   = c(5000, 5000),
  importance = TRUE
)
# graph the output on accuracy score
varImpPlot(rf.sim2, type = 1)
# Theoretically meaningful variables - highlighted regardless of Gini rank
highlight_vars <- c(
  "PERFORM_CNS_SCORE",
  "PERFORM_CNS_SCORE_DESCRIPTION",
  "stress_score",
  "recent_delinquency_flag",
  "pri_any_overdue_flag",
  "DELINQUENT_ACCTS_IN_LAST_SIX_MONTHS",
  "credit_history_months",
  "avg_acct_age_months",
  "ltv_high_flag",
  "pri_leverage_ratio"
)

# Pull importance scores from model object
imp_df <- importance(rf.sim2) |>
  as.data.frame() |>
  rownames_to_column("Feature") |>
  arrange(MeanDecreaseGini) |>
  mutate(
    Feature    = factor(Feature, levels = Feature),
    Highlighted = ifelse(Feature %in% highlight_vars, "Key predictors", "Other")
  )

ggplot(imp_df, aes(x = MeanDecreaseGini, y = Feature, fill = Highlighted)) +
  geom_col() +
```

```
  scale_fill_manual(
    values = c("Key predictors" = "steelblue", "Other" = "grey75"),
    name   = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    axis.text.y      = element_text(size = 8),
    panel.grid.major.y = element_blank(),
    panel.grid.minor   = element_blank(),
    legend.position  = "top",
    plot.title       = element_text(size = 12)
  ) +
  labs(
    title = "RF2 Variable Importance",
    x     = "Mean Decrease Gini",
    y     = NULL
  )
    # pred.rf.sim2 = rf.sim2$predicted
    # table(training[,41], pred.rf.sim2)
  #
  #    pred.rf.sim2
  #       0     1
  # 0 39958 17561
  # 1  6416  8195


  # pred.val.rf.sim2 <- predict(
  #   rf.sim2,
  #   newdata = x_validation,
  #   type = "class"
  # )
  #
  # table(validation[,41], pred.val.rf.sim2)
 #
# pred.val.rf.sim2
#        0    1
#   0 8611 3898
#   1 1396 1738
rf_results <- tribble(
  ~Model, ~Dataset, ~TN, ~FP, ~FN, ~TP,

  "rf.sim2", "Training",   39958, 17561, 6416, 8195,
```

```
  "rf.sim2", "Validation",  8611,  3898, 1396, 1738,
  "rf.sim2", "Test",        8618,  3927, 1292, 1759
) %>%
  mutate(
    Accuracy    = (TP + TN) / (TP + TN + FP + FN),
    Recall      = TP / (TP + FN),
    Precision   = TP / (TP + FP),
    Specificity = TN / (TN + FP),
    F1          = 2 * Precision * Recall / (Precision + Recall)
  )

rf_results %>%
  mutate(
    across(
      c(Accuracy, Recall, Precision, Specificity, F1),
      ~ scales::percent(.x, accuracy = 0.1)
    )
  ) %>%
  select(
    Model,
    Dataset,
    Accuracy,
    Recall,
    Precision,
    Specificity,
    F1
  ) %>%
  kable(
    caption = "Random Forest Performance Metrics"
  )
# https://www.statology.org/random-forest-in-r/
# Trying to optimize with RFTune
# Commented out to allow the doc to render faster
tune.rf <- tuneRF(
  x = training[, drop_cols],

  y = factor(training$LOAN_DEFAULT),

  stepFactor = 1.5,
  improve = 0.01,
  ntreeTry = 300
)
```

```r
# Commented out to allow the doc to render faster
 best.mtry <- tune.rf[which.min(tune.rf[,2]),1]
#
rf.final <- randomForest(
  x = training[, drop_cols],
  y = factor(training$LOAN_DEFAULT),

  ntree = 1000,
  mtry = best.mtry,

  importance = TRUE,

  sampsize = c(10000, 10000)
)
 # # Commented out to allow the doc to render faster
   pred.rf.final = rf.final$predicted
   table(training[,41], pred.rf.final)
   # pred.rf.final
 #   #       0     1
 #   # 0 43542 13977
 #   # 1  7518  7093
 #
   pred.rf.final <- predict(
   rf.final,
   newdata = x_validation,
   type = "class"
 )
 #
 #
  table(validation[,41], pred.rf.final)
#
#   # pred.rf.final
#   #      0    1
#   # 0 9345 3164
#   # 1 1646 1488
# Pulling everything from the commented out models above.
rf_results <- tribble(
  ~Model, ~Dataset, ~TN, ~FP, ~FN, ~TP,

  "rf.sim1", "Training",   57124, 395, 14256, 355,
  "rf.sim1", "Validation", 12415, 94, 3056, 78,
```

```
  "rf.sim2", "Training", 39958, 17561, 6416, 8195,
  "rf.sim2", "Validation", 8613, 3896, 1397, 1737,

  "rftune", "Training",  43542, 13977, 7518, 7093,
  "rftune", "Validation", 9345, 3164, 1646, 1488
) %>%
  mutate(
    Accuracy = (TP + TN) / (TP + TN + FP + FN),

    Recall = TP / (TP + FN),

    Precision = TP / (TP + FP),

    Specificity = TN / (TN + FP),

    F1 = 2 * (Precision * Recall) / (Precision + Recall)
  )

kable(
  rf_results %>%
    mutate(
      across(
        c(Accuracy, Recall, Precision, Specificity, F1),
        round,
        3
      )
    ),
  caption = "Random Forest Model Comparison"
)
# # XG Boost
#
# # convert to matrices
x_train <- model.matrix(
  ~ . - 1,
  data = training[, drop_cols]
)

x_valid <- model.matrix(
  ~ . - 1,
  data = validation[, drop_cols]
)
```

```r
x_test <- model.matrix(
  ~ . - 1,
  data = testing[, drop_cols]
)



y_train <- training$LOAN_DEFAULT
y_valid <- validation$LOAN_DEFAULT
y_test <- testing$LOAN_DEFAULT


# Deal with the imbalance
scale_pos_weight <- sum(y_train == 0) / sum(y_train == 1)

scale_pos_weight


# Training matrices
dtrain <- xgb.DMatrix(
  data = x_train,
  label = y_train
)

dvalid <- xgb.DMatrix(
  data = x_valid,
  label = y_valid
)

dtest <- xgb.DMatrix(
  data = x_test,
  label = y_test
)

# Initial XGBoost model
params <- list(
  objective = "binary:logistic",
  eval_metric = "auc",
  max_depth = 6,
  eta = 0.05,
  subsample = 0.8,
  colsample_bytree = 0.8,
```

```
  scale_pos_weight = scale_pos_weight
)

xgb_model <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 300,
  watchlist = list(train = dtrain, validation = dvalid),
  #verbose = 1
)


# # Get predictions
 pred_prob <- predict(xgb_model, dvalid)

 pred_class <- ifelse(pred_prob > 0.30, 1, 0)

 conf_mat <- table(
   Actual = y_valid,
   Predicted = pred_class
 )

 kable(as.data.frame.matrix(conf_mat),
       caption = "Confusion Matrix")


# Confusion Matrix
# 0 1
# 0 11792   717
# 1 2971    163
conf_xgb <- matrix(
  c(11792, 717,
    2971, 163),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Actual = c("0", "1"),
    Predicted = c("0", "1")
  )
)

knitr::kable(
```

```
  conf_xgb,
  caption = "XGBoost Validation Confusion Matrix"
)
# roc_obj <- roc(y_valid, pred_prob)
#
# auc(roc_obj)

# Area under the curve: 0.4997
thresholds <- c(0.10, 0.15, 0.20, 0.25, 0.30)

xgb_threshold_results <- lapply(thresholds, function(t) {
  pred_class <- ifelse(pred_prob > t, 1, 0)
  tab <- table(
    Actual = y_valid,
    Predicted = pred_class
  )

  TN <- tab["0", "0"]
  FP <- tab["0", "1"]
  FN <- tab["1", "0"]
  TP <- tab["1", "1"]

  data.frame(
    Threshold = t,
    Accuracy = (TP + TN) / sum(tab),
    Recall = TP / (TP + FN),
    Precision = TP / (TP + FP),
    Specificity = TN / (TN + FP),
    F1 = 2 * ((TP / (TP + FP)) * (TP / (TP + FN))) /
      ((TP / (TP + FP)) + (TP / (TP + FN)))
  )
}) |>
  bind_rows()

knitr::kable(
  xgb_threshold_results,
  format      = "simple",
  align       = c("c", "r", "r", "r", "r", "r"),
  caption     = "XGBoost Peformance by Threshold"
)

#
```

```
# XGBoost Peformance by Threshold
# Threshold Accuracy    Recall  Precision   Specificity F1
# 0.10  0.2075050   0.9875558   0.2002847   0.0120713   0.3330285
# 0.15  0.2922074   0.8567326   0.2017583   0.1507714   0.3266026
# 0.20  0.4821965   0.5402042   0.2027059   0.4676633   0.2947937
# 0.25  0.6723135   0.2115507   0.1998192   0.7877528   0.2055177
# 0.30  0.7642396   0.0520102   0.1852273   0.9426813   0.0812157
xgb_threshold_results <- tribble(
  ~Threshold, ~Accuracy, ~Recall, ~Precision, ~Specificity, ~F1,
  0.10, 0.2075050, 0.9875558, 0.2002847, 0.0120713, 0.3330285,
  0.15, 0.2922074, 0.8567326, 0.2017583, 0.1507714, 0.3266026,
  0.20, 0.4821965, 0.5402042, 0.2027059, 0.4676633, 0.2947937,
  0.25, 0.6723135, 0.2115507, 0.1998192, 0.7877528, 0.2055177,
  0.30, 0.7642396, 0.0520102, 0.1852273, 0.9426813, 0.0812157
)

xgb_threshold_results %>%
  mutate(
    across(
      c(Accuracy, Recall, Precision, Specificity, F1),
      ~ scales::percent(.x, accuracy = 0.1)
    )
  ) %>%
  kable(
    caption = "XGBoost Performance by Classification Threshold"
  )

# Lets see how it does on our test data.
# # Test Data on random forest 1
  pred.test.rf.sim1 <- predict(
   rf.sim1,
   newdata = testing[drop_cols],
   type = "class"
 )
 table(testing[,41], pred.test.rf.sim1)
conf_rf1_test <- matrix(
  c(8567, 3978,
    1269, 1782),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Actual = c("0", "1"),
```

```
    Predicted = c("0", "1")
  )
)

knitr::kable(
  conf_rf1_test,
  caption = "Random Forest Model 1 Test Set Confusion Matrix"
)
# Test Data on random forest 2 (Paul's improved model)
  pred.test.rf.sim2 <- predict(
    rf.sim2,
    newdata = testing[drop_cols],
    type = "class"
  )
  table(testing[,41], pred.test.rf.sim2)
# Static table of test Data on random forest 2 (Paul's improved model)
conf_rf2_test <- matrix(
  c(8618, 3927,
    1292, 1759),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Actual = c("0", "1"),
    Predicted = c("0", "1")
  )
)

knitr::kable(
  conf_rf2_test,
  caption = "Random Forest Model 2 Test Set Confusion Matrix"
)
# # XGB Score on the test data
 test_prob <- predict(xgb_model, dtest)

 test_class <- ifelse(test_prob > 0.30, 1, 0)

 #
 conf_mat <- table(
   Actual = y_test,
   Predicted = test_class
 )
```

```
 #
 kable(as.data.frame.matrix(conf_mat),
       caption = "Confusion Matrix")
# static table of test data on XGB
conf_xgb_test <- matrix(
  c(11731, 814,
    2902, 149),
  nrow = 2,
  byrow = TRUE,
  dimnames = list(
    Actual = c("0", "1"),
    Predicted = c("0", "1")
  )
)

knitr::kable(
  conf_xgb_test,
  caption = "XGBoost Test Set Confusion Matrix"
)
```